

Parallel & Concurrent Programming in Python

Concepts, Libraries, Examples, Advantages & Limitations

Learning Objectives

- Understand concurrency vs parallelism
- Learn Python libraries for parallel & concurrent execution
- Identify when to use each approach
- Compare tools based on performance and use cases

Concurrency vs Parallelism

Concurrency

- Multiple tasks make progress together - Tasks may interleave - Not necessarily simultaneous

Parallelism

- Multiple tasks execute at the same time - Requires multiple cores / processors

Python Execution Model

- Python uses **Global Interpreter Lock (GIL)**
- GIL limits true parallelism in threads
- Multiprocessing bypasses GIL using separate processes

Global **Interpreter Lock (GIL)** is a **mutex (lock)** used by **CPython** (the standard Python implementation) to ensure that **only one thread executes Python bytecode at a time**, even on multi-core CPUs.

threading Module

What is threading?

- Runs multiple threads in one process - Shared memory

Example

```
from threading import Thread
```

```
def task():  
    print("Hello")
```

```
Thread(target=task).start()
```

threading – Advantages & Limitations

Advantages - Lightweight - Easy communication (shared memory) - Best for I/O-bound tasks

Limitations - No true CPU parallelism (GIL) - Race conditions possible

multiprocessing.Process

What is Process?

- Creates independent OS-level processes - True parallelism

Example

```
from multiprocessing import Process
```

```
def task():  
    print("Running in process")
```

```
Process(target=task).start()
```

Process – Advantages & Limitations

Advantages - True CPU parallelism - No GIL limitation

Limitations - Higher memory usage - Manual result sharing needed

multiprocessing.Pool

What is Pool?

- Manages a pool of worker processes - Automatic task distribution

Example

```
from multiprocessing import Pool
```

```
with Pool(4) as p:  
    print(p.map(lambda x: x*x, [1,2,3,4]))
```

Pool – Advantages & Limitations

Advantages - Simple syntax - Automatic load balancing

Limitations - Less control over workers - All tasks must be similar

concurrent.futures Module

Unified High-Level API

- Cleaner alternative to threading & multiprocessing

Two Executors: - ThreadPoolExecutor - ProcessPoolExecutor

ThreadPoolExecutor

Example

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor() as ex:
    ex.map(print, [1,2,3])
```

Use Case: I/O-bound tasks

ProcessPoolExecutor

Example

```
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor() as ex:
    print(list(ex.map(lambda x: x*x, [1,2,3])))
```

Use Case: CPU-bound tasks

asyncio – Asynchronous Programming

What is asyncio?

- Single-threaded, event-driven concurrency - Uses async/await

Example

```
import asyncio
```

```
async def task():
    await asyncio.sleep(1)
```

asyncio – Advantages & Limitations

Advantages - Handles thousands of I/O tasks - Very low overhead

Limitations - No CPU parallelism - Steeper learning curve

joblib

What is joblib?

- Simplified parallelism for ML

Example

```
from joblib import Parallel, delayed

Parallel(n_jobs=2)(delayed(print)(i) for i in range(4))
```

numba

What is numba?

- JIT compiler for numerical code - Automatic parallel loops

Example

```
from numba import njit, prange

@njit(parallel=True)
def f(a):
    for i in prange(len(a)):
        a[i] += 1
```

GPU-Based Parallelism

- CUDA
- PyTorch
- TensorFlow

Best for - Deep learning - Massive data parallelism

Comparison Table

Tool	Parallel	Best For
------	----------	----------

threading	No	I/O-bound
Process	Yes	Few CPU tasks
Pool	Yes	Many similar tasks
ThreadPoolExecutor	No	I/O
ProcessPoolExecutor	Yes	CPU
asyncio	No	Massive I/O
joblib	Yes	ML
numba	Yes	Numerical

Choosing the Right Tool

- I/O-bound → threading / asyncio
- CPU-bound → multiprocessing / ProcessPoolExecutor
- ML / Numeric → joblib / numba
- Big Data → Dask / Ray